COMP5423 Group Project Instruction

Topic	Skill-Augmented Search Agent
Group Size	4 students per group (two groups of 3 students are allowed)
Due Date	**November 26, 2026 (Hong Kong time, 23:59)**
TA	Runyang You (25093051r@connect.polyu.hk)
	Hongru Cai (25145475r@connect.polyu.hk)
	Zhenning Shi (26035798r@connect.polyu.hk)

Objective

This project aims to develop a **skill-augmented search agent** that answers challenging questions using a fixed document collection. The system should combine retrieval, large language model (LLM) reasoning, tool use, and reusable skills in one end-to-end workflow.

Unlike a single-turn retrieval-augmented generation (RAG) pipeline, the agent should be able to perform multiple search steps. It may reformulate a query, call search-support tools, inspect documents, collect evidence, apply relevant skills, verify an answer, and decide when to stop.

All groups must use the same specified LLM backbone through an application programming interface (API). The project does not require training or fine-tuning a large language model. Students should improve the system through retrieval design, tool coordination, agent workflow, context management, and skill design. Representative papers on search agents and agentic deep research are listed in References [12–14].

Project Resources

The project will use BrowseComp-Plus [1], a benchmark for evaluating search agents over a fixed collection of 100,195 documents. A validation set of 100 questions sampled from the original dataset, with answers, is available on Hugging Face. A separate test set of 100 questions will be released 2 weeks before the submission deadline, without answers, for the final performance evaluation. Both sets will use the same document collection.

Split	Number	Released Fields
Validation	100	ID, question, answer
Test	100	ID, question
Document Collection	100,195	Document ID, text, source URL

Table 1: Planned structure of the course dataset.

The official backbone is `DeepSeek-V4-Flash` [18]. Several API service options are listed in References [27–31]. Students may use other API services if they provide the same model and functionality.

Task Overview

The project is divided into four components, as summarized in Table 2.

Component	Weight (%)
System Implementation	40
Written Report	20
Demo Video	20
System Performance	20

Table 2: Overall project component weighting.

Detailed Requirements

1 System Implementation (40%)

The system should contain retrieval methods, multiple search-support tools, a tool-using search agent, reusable skills, and a simple user interface. The same implementation should be able to run on the released validation and test files without manually editing individual questions or answers.

1.1 Retrieval Methods and Tool Design (12%)

Retrieval Methods (8%) Students are required to implement retrieval methods from at least one of the following categories:

- sparse retrieval (e.g., TF–IDF, BM25 [2], or predicted term weights from neural models);
- dense retrieval using encoder-based models (e.g., E5 [4], BGE [5], or GTE [6]); and
- instruction-aware dense retrieval using LLM-based embedding models (e.g., Qwen3-Embedding [7] or E5-Mistral [8]).

Example implementation resources include BM25S [3], FAISS [9], and FlagEmbedding [17]. All implemented retrieval methods must be exposed to the agent through one unified retrieval tool. The tool should accept a search query and return ranked document identifiers, scores, and text snippets.

Students may combine multiple retrieval methods through fusion, internal routing, or a method-selection parameter. Regardless of the internal design, the agent must be given exactly one retrieval tool and must not receive separate tools for different retrievers.

Grading rule. Implementing retrieval from one category earns 4 points, implementing retrieval from two categories earns 6 points, and implementing retrieval from all three categories earns 8 points. Multiple methods from the same category count as one category for this rule.

Tool Design (4%) In addition to the unified retrieval tool, the agent should have at least one other type of supporting tool. A supporting tool may, for example, open a document, find text within a document, inspect metadata, or filter search results. Examples are available in References [19,20].

Each tool should have a clear name, function, input, and output, and it should be callable by the agent. The report should list the total number of tool types and describe each tool.

Grading rule. Providing only the unified retrieval tool earns 2 points. Providing the unified retrieval tool and at least one additional type of supporting tool earns 4 points. Only tools that are implemented and callable by the agent are counted.

1.2 Search Agent Design (14%)

The system must use the specified LLM backbone through an API. The Basic Tool-Using Agent is required. Feature A and Feature B are optional extensions, and each feature receives its allocated score only when it is implemented. Implementing the basic component and both features results in full marks for this part. Whether a feature has been implemented will be determined from the written report, demo video, and submitted code.

Basic Tool-Using Agent (4%) The basic agent should receive a question, select an appropriate tool from the available tool set, generate its input, execute the tool, and use the returned information to produce the answer.

Feature A: Iterative Agent Loop (5%) The agent should go beyond a fixed single-pass pipeline and support an iterative agent loop in which later actions can depend on earlier search results.

For example, the agent may reformulate or decompose a query, call the retrieval tool again, inspect additional documents, check a candidate answer, or decide when enough evidence has been collected.

Students may design other iterative workflows. A fixed retrieval-then-generation pipeline without result-dependent iteration is not considered a complete iterative agent loop. Possible designs are illustrated by ReAct and agent-loop tutorials [10,21,22].

Feature B: Context Management (5%) The agent should maintain a structured internal state separately from the raw conversation history. The state should record visited documents, selected evidence, and the current search progress, such as active sub-queries or a candidate answer.

The system should implement at least one method for controlling the context passed to the LLM, such as deduplication, selection, pruning, compression, or summarization. Simply appending all previous messages and tool outputs to the prompt is not considered complete context management. State, trimming, summarization, and memory examples are available in References [11,23,24].

1.3 Reusable Skills (10%)

A skill is a reusable procedure that guides the agent in a defined situation, such as decomposing a multi-clue question, resolving an ambiguous entity, or recovering from an unsuccessful search.

Basic Skill Structure (6%) Each group must implement at least one skill that follows the Agent Skills specification [15]. Students may study existing skills as references, but they must not copy, directly reuse, or submit a skill created by others. Each skill must be stored as a directory with the following standard structure:

```
skill-name/
|-- SKILL.md          # required
|-- scripts/          # optional
|-- references/       # optional
`-- assets/           # optional
```

The required `SKILL.md` file must contain YAML frontmatter with at least `name` and `description`, followed by Markdown instructions. The skill directory may include scripts, references, assets, or other supporting files when needed.

The full skill instructions must not be included permanently in the base prompt. The agent should load the skill directory at runtime and follow its instructions.

Bonus: Automatic Skill Selection (4%) To earn the bonus score, the system must provide at least two skills with different functions. The agent should automatically decide which skill to load based on the current question or search state, rather than requiring the user to specify a skill. The agent may also decide that no skill is needed. The report and demo should show different questions that trigger different skill-selection decisions.

1.4 User Interface (4%)

The interface may be terminal-based or web-based. It should allow a user to enter a question and inspect the final answer, supporting evidence, selected skills, and tool-use process. A clear and readable interface is sufficient; visual complexity is not required.

2 Written Report (20%)

All groups must use the report template on Overleaf: https://www.overleaf.com/read/wdbsnbtqwcrv#5382c5. The report must contain at least four pages, excluding references and the Contribution and Declaration sections, and must be submitted in PDF format.

The final report must retain and complete the **Contribution and Declaration** section at the end of the template. The contribution percentages must sum to 100%, and every group member must provide a signature. A report with an incomplete Contribution and Declaration section will not be accepted.

The report should include:

- **System Design Methodology:** describe the overall design and key components of the system.
- **System Flowchart:** provide a labeled diagram showing the workflow from question input to final answer.
- **Result Analysis:** analyze the experimental results under different system configurations, with relevant tables or figures.
- **User Interface Design:** provide screenshots and explain how users interact with the system.
- **Additional Exploration (Optional):** describe any additional experiments, features, or optimization attempts.

3 Demo Video (20%)

The demo video should be no longer than five minutes and should be submitted in MP4. The video should:

- demonstrate the system in action and highlight the main challenges addressed;
- show any features or bonuses for which the group claims credit; and
- provide a brief overview of the code structure and explain the role of its main components.

The purpose of the video is to show that the group understands the submitted system and that the reported behavior is produced by the implementation.

4 System Performance (20%)

This component awards 10 points for a valid submission and up to 10 points for relative performance. Submit `test_prediction.jsonl` with one JSON object per test question, using the fields below. See the sample file.

```
{
  "id": <str>,
  "question": <str>,
  "answer": <str>,
  "trajectory": [<agent_step>, ...],
  "tool_calls": <int>,
  "total_tokens": <int>
}
```

For each question, `trajectory` must record the complete agent process in chronological, machine-readable form. `tool_calls` counts all calls issued by the agent, excluding internal operations within a tool. `total_tokens` is the sum of the API's `usage.total_tokens` over all LLM calls for that question.

Valid System Submission (10%) A group receives 10 points if its system satisfies the Basic Tool-Using Agent requirement, runs end-to-end, and generates exactly one valid record with all required fields for each of the 100 test questions. Incorrect answers do not reduce these 10 points. Submissions that do not meet these requirements, or contain manually constructed predictions or trajectories, receive 0 points for the entire System Performance component and are excluded from the relative performance ranking.

Relative Performance (10%) Answer accuracy contributes up to 6 points, tool-call efficiency up to 2 points, and token efficiency up to 2 points. Accuracy is the percentage of test answers that match the references after lowercasing, removing punctuation and English articles (*a*, *an*, and *the*), and normalizing whitespace (normalized EM). Efficiency uses the total tool calls and total API tokens summed across all 100 test questions. Higher EM and fewer calls or tokens are better.

For each metric, eligible groups are ranked from best to worst and assigned evenly spaced percentile scores from 100 to 0. Tied groups receive the average of the scores for their tied positions. The three percentile scores are denoted by P_{EM}, P_{calls}, and P_{tokens}, respectively. The total score is calculated directly as follows and rounded to two decimal places:

$$\text{System Performance Score} = 10 + 6\frac{P_{\text{EM}}}{100} + 2\frac{P_{\text{calls}}}{100} + 2\frac{P_{\text{tokens}}}{100}.$$

For example, a valid submission with percentile scores of 80 for EM, 60 for tool calls, and 40 for tokens receives 16.8 out of 20 points, as shown in Table 3.

Component	Maximum Points	Example Percentile	Points Earned
Valid System Submission	10	–	10
Answer Accuracy	6	80	4.8
Tool-Call Efficiency	2	60	1.2
Token Efficiency	2	40	0.8
Total	**20**	–	**16.8**

Table 3: Example calculation of the System Performance score.

Submission

Submit one compressed file for the entire group through Canvas. The recommended directory structure is:

```
Group-X.zip
|-- test_prediction.jsonl
|-- code/
|   |-- README.md
|   |-- requirements.txt or environment.yml
|   |-- skills/
|   |   `-- skill-name/
|   |       |-- SKILL.md
|   |       |-- scripts/ (optional)
|   |       |-- references/ (optional)
|   |       `-- assets/ (optional)
|   |-- main program and supporting source files
|-- Group-X-Report.pdf
`-- Group-X-Video.mp4
```

1. Prediction File with Embedded Trajectories Include `test_prediction.jsonl` in exactly the required format. Each JSON object must contain the complete trajectory for its corresponding test question. No separate trajectory file or directory is required.

2. Code Directory The `code/` directory should contain all source files needed to run the system. The README should explain:

- the code structure;
- environment setup and dependency versions;
- how to reproduce the `test_prediction.jsonl` file from the test questions; and
- how the LLM API key should be supplied.

Do not include API keys, passwords, model weights, or other credentials in the submission. Large indexes or model files may be shared through a public read-only cloud link if necessary.

3. Report and Video Include the final report as `Group-X-Report.pdf` and the demo as `Group-X-Video.mp4`.

4. Group Submission Each student must join the corresponding group on Canvas. Groups have four members, except for two designated three-member groups. Only one member per group needs to submit the final package.

> **Note.** Submissions that fail to follow the required directory structure or file format may not be evaluated correctly. Please test the packaged submission locally before uploading it to ensure that the system can be executed end-to-end.

Late Submission Policy. Late submissions will incur a penalty of 10% of the total score per day. Submissions more than 3 days late will not be accepted and will receive a score of 0.

Academic Integrity and Verification

All submitted answers and embedded trajectories must be generated by the submitted system. Hard-coded test answers, question-to-answer lookup tables, manual editing of individual predictions, or direct use of hidden labels are not allowed.

The teaching team may inspect source code, compare predictions and embedded trajectories, ask questions about specific implementation choices, or rerun selected cases when necessary. Groups must keep the system environment and relevant API configuration available until grading is complete.

References

Papers

[1] Zijian Chen, Xueguang Ma, Shengyao Zhuang, et al. 2026. BrowseComp-Plus: A Fair and Disentangled Evaluation Benchmark for Deep Search Agents. ACL 2026.

[2] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval.

[3] Xing Han Lù. 2024. BM25S: Orders of Magnitude Faster Lexical Search via Eager Sparse Scoring.

[4] Liang Wang, Nan Yang, Xiaolong Huang, et al. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training.

[5] Shitao Xiao, Zheng Liu, Peitian Zhang, et al. 2024. C-Pack: Packed Resources for General Chinese Embeddings. SIGIR 2024.

[6] Zehan Li, Xin Zhang, Yanzhao Zhang, et al. 2023. Towards General Text Embeddings with Multi-stage Contrastive Learning.

[7] Yanzhao Zhang, Mingxin Li, Dingkun Long, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.

[8] Liang Wang, Nan Yang, Xiaolong Huang, et al. 2024. Improving Text Embeddings with Large Language Models. ACL 2024.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale Similarity Search with GPUs. IEEE Transactions on Big Data, 2021.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[11] Charles Packer, Vivian Fang, Shishir G. Patil, et al. 2023. MemGPT: Towards LLMs as Operating Systems.

[12] Weizhi Zhang, Yangning Li, Yuanchen Bei, et al. 2025. From Web Search towards Agentic Deep Research: Incentivizing Search with Reasoning Agents.

[13] Xiaoxi Li, Guanting Dong, Jiajie Jin, et al. 2025. Search-o1: Agentic Search-Enhanced Large Reasoning Models. EMNLP 2025.

[14] Yuxuan Huang, Yihang Chen, Haozheng Zhang, et al. 2025. Deep Research Agents: A Systematic Examination and Roadmap.

Resources

[15] Agent Skills. Agent Skills Specification.

[16] Texttron. BrowseComp-Plus Code and Data.

[17] FlagOpen. FlagEmbedding: Retrieval and Reranking Toolkit.

[18] DeepSeek-AI. 2026. DeepSeek-V4-Flash Model Card.

[19] LangChain. Tools.

[20] LangChain. Deep Agents Filesystem Tools.

[21] LangChain. Agents and Tool-Calling Loops.

[22] LangChain. LangGraph Graph API: State, Branches, and Loops.

[23] LangChain. Short-Term Memory.

[24] Anthropic. 2025. Effective Context Engineering for AI Agents.

[25] Weizhi Zhang, et al. Awesome Deep Research.

[26] RUC-NLPIR. Search-o1.

APIs

[27] DeepSeek. Chat Completions API.

[28] OpenRouter. DeepSeek-V4-Flash API.

[29] Alibaba Cloud Model Studio. DeepSeek API.

[30] SiliconFlow. Chat Completions API.

[31] Volcano Engine. Ark API Documentation.